chapter 8. If you need to obtain dependencies to create an instance of your service, you can reach into the container at runtime in this way, but you should avoid doing so if possible.

TIP Avoid calling `GetService<T>()` and `GetRequiredService<T>()` in your factory functions if possible. Instead, favor constructor injection; it's more performant and simpler to reason about.

Open generics and dependency injection

As already mentioned, you couldn't use the generic registration methods with `EmailServerSettings` because it uses primitive dependencies (in this case, `string` and `int`) in its constructor. Neither can you use the generic registration methods to register open generics.

Open generics are types that contain a generic type parameter, such as `Repository <T>`. You normally use this sort of type to define a base behavior that you can use with multiple generic types. In the `Repository<T>` example, you might inject `IRepository <Customer>` into your services, which should inject an instance of `DbRepository <Customer>`, for example.

To register these types, you must use a different overload of the `Add*` methods, as in this example:

```
builder.Services.AddScoped(typeof(IRespository<>), typeof(DbRepository<>));
```

This code ensures that whenever a service constructor requires `IRespository<T>`, the container injects an instance of `DbRepository<T>`.

At this point, all your dependencies are registered. But your Program.cs is starting to look a little messy, isn't it? The choice is entirely down to personal preference, but I like to group my services into logical collections